

■ Koki Woki ChefBot

Technical Reference Report

Objects · Built-in Functions · Architecture

Scala Functional Chatbot — Full Codebase Analysis

Files: ChatBotInteract · ConversationBrain · CookingChatBot ·
FormattedResponse · Login · Preferences · Recipes · Recommendations

Table of Contents

- 1. Project Overview
- 2. Architecture & Package Layout
- 3. Object Reference — Roles & Responsibilities
 - 3.1 RecipeKnowledgeBase (chatBot.data)
 - 3.2 ConversationBrain (chatBot.brain)
 - 3.3 PreferenceManager (chatBot.engine)
 - 3.4 RecommendationEngine (chatBot.recommend)
 - 3.5 ResponseFormatter (chatBot.response)
 - 3.6 UserAuth (chatBot.auth)
 - 3.7 Chatbot (chatBot.ui)
 - 3.8 startChefBot — @main entry point
- 4. Built-in & Standard-Library Functions — Complete Index
 - 4.1 Collection Operations
 - 4.2 String Operations
 - 4.3 Option Combinators
 - 4.4 I/O & OS Operations (os-lib)
 - 4.5 Control Flow & Pattern Matching
- 5. Data Flow Diagram
- 6. Key Design Patterns

1. Project Overview

Koki Woki ChefBot is a terminal-based conversational cooking assistant written in Scala. It allows users to register, log in, maintain session history, state dietary preferences, and receive personalised recipe suggestions. The system is split across eight source files covering authentication, recipe data, preference persistence, NLP-lite conversation analysis, response formatting, recommendation logic, and the main interactive loop.

File	Package	Primary Role
CookingChatBot.scala	(top-level)	@main entry point, auth loop, chat loop
Recipes.scala	chatBot.data	Recipe case class + 46-recipe knowledge base
ConversationBrain.scala	chatBot.brain	NLP intent/sentiment/topic analysis
Preferences.scala	chatBot.engine	Disk-backed user preference store
Recommendations.scala	chatBot.recommend	Personalised recipe recommendation engine
FormattedResponse.scala	chatBot.response	All user-visible text formatting
LogIn.scala	chatBot.auth	Registration, login, session persistence
ChatBotInteract.scala	chatBot.ui	Core response generator, history manager

2. Architecture & Package Layout

The codebase follows a layered architecture with strict upward dependency flow. Higher layers depend on lower ones; no circular imports exist.

Layer 1 — Entry Point	CookingChatBot (@main)
Layer 2 — UI / Engine	Chatbot object (chatBot.ui)
Layer 3 — Services	UserAuth · PreferenceManager · RecommendationEngine · ResponseFormatter
Layer 4 — Intelligence	ConversationBrain
Layer 5 — Data	RecipeKnowledgeBase · Recipe

The dependency rule: every lower layer is unaware of the layers above it. RecipeKnowledgeBase has zero imports from other chatBot packages; ConversationBrain depends only on chatBot.data; services depend on data + brain; the entry point depends on everything.

3. Object Reference — Roles & Responsibilities

3.1 RecipeKnowledgeBase (chatBot.data — Recipes.scala)

Recipes.scala

The single source of truth for all recipe data. It holds the 46-recipe catalogue and pre-computed lookup lists used throughout the system for detection and filtering. The companion case class Recipe is also defined here.

Member / Method	Signature / Type	Description
<code>recipes</code>	<code>List[Recipe] (val)</code>	All 46 recipes as immutable list
<code>allCuisines</code>	<code>List[String] (val)</code>	Distinct cuisine names in lowercase, derived from recipes
<code>allTags</code>	<code>List[String] (val)</code>	Distinct dietary tags in lowercase, derived from recipes
<code>allDifficulties</code>	<code>List[String] (val)</code>	Distinct difficulty levels in lowercase, derived from recipes

3.2 ConversationBrain (chatBot.brain — ConversationBrain.scala)

ConversationBrain.scala

The NLP and memory layer of the chatbot. Analyses user messages for intent, sentiment, cuisine mentions, dietary tags and difficulty cues. Also handles chat history persistence and smart search / autocomplete.

Member / Method	Signature / Type	Description
<code>detectIntent(userInput)</code>	<code>String => String</code>	Returns Positive_Interest, Negative_Feedback, or General_Query
<code>isNegativeSentiment(input)</code>	<code>String => Boolean</code>	Convenience wrapper — true when intent is Negative_Feedback
<code>getUserMood(history)</code>	<code>List[InteractionEntry] => String</code>	Compares positive vs negative count; returns Positive/Negative/Neutral
<code>extractTopics(history)</code>	<code>List[IE] => List[String]</code>	Finds recipe names, cuisines, and tags mentioned in history
<code>getMostDiscussedTopics(history)</code>	<code>List[IE] => List[(String, Int)]</code>	Counts cuisine frequency in history, sorted descending
<code>detectRepeatedQuery(input, history)</code>	<code>(String, List[IE]) => Boolean</code>	True if identical trimmed input already exists in history
<code>detectCuisine(input)</code>	<code>String => Option[String]</code>	Matches any known cuisine in the input string
<code>detectTag(input)</code>	<code>String => Option[String]</code>	Matches any known dietary tag in the input string
<code>detectDifficulty(input)</code>	<code>String => Option[String]</code>	Matches any known difficulty level in the input string

saveChat(user, num, history)	(String, Int, List[IE])=>Unit	Persists history to Users-data//Chat.txt
loadChat(user, num)	(String, Int)=>List[IE]	Loads history from disk; returns Nil if absent
smartRecipeSearch(input)	String => List[Recipe]	Keyword search across name, cuisine, difficulty, ingredients, tags
prefixMatches(input, candidates)	(String, List[String])=>List[String]	Autocomplete: returns candidates starting with input
generateSuggestions(input)	String => List[String]	Merges prefix-matched recipes, cuisines, ingredients, tags (top 5)

3.3 PreferenceManager (chatBot.engine — Preferences.scala)

Preferences.scala

Manages user preferences as a disk-backed key-value store. Each user has a Preferences.txt with lines like cuisine:italian. The store is read fresh on every call — no in-memory cache — ensuring multi-session consistency.

Member / Method	Signature / Type	Description
loadPrefs(user)	String => Map[String, String]	Reads Preferences.txt and parses into a Map
storePref(user, k, v)	(String, String, String)=>Unit	Merges one key-value into the existing map and overwrites file
getPref(user, key)	(String, String)=>Option[String]	Convenience: reads one key from the loaded map
clearPref(user, key)	(String, String)=>Unit	Removes a key from the map and rewrites the file

3.4 RecommendationEngine (chatBot.recommend — Recommendations.scala)

Recommendations.scala

Generates up to 5 personalised recipe suggestions using a progressive-relaxation strategy. The engine first tries to match all active filters (cuisine + tag + difficulty), then drops constraints one by one until a non-empty result is found.

Member / Method	Signature / Type	Description
recommend(user, contextCuisine)	(String, Option[String]) => List[Recipe]	Main entry point. Loads prefs, applies progressive filter, shuffles, returns top 5

3.5 ResponseFormatter (chatBot.response — FormattedResponse.scala)

FormattedResponse.scala

Pure formatting object — all methods are stateless string transformations. Centralises every piece of user-visible text, making it easy to reskin or localise.

Member / Method	Signature / Type	Description
greetingResponse(user)	String => String	Welcome message addressed by username
formatRecipe(r)	Recipe => String	Full recipe card: ingredients, steps, metadata
formatList(title, rs)	(String, List[Recipe])=>String	Lists recipes with difficulty and prep time
showGuide()	() => String	Full command reference / help text
repeatedQueryNote()	() => String	Banner appended when query is a repeat
preferenceSaved(key, value)	(String, String)=>String	Confirmation message when a preference is stored

3.6 UserAuth (chatBot.auth — Login.scala)

Login.scala

Handles all file-system operations related to user accounts. Each user gets a dedicated directory under Users-data/ containing a password file, Summaries.txt, chat transcript files, and Preferences.txt.

Member / Method	Signature / Type	Description
userExists(name)	String=>Boolean	Checks whether the user directory exists on disk
register(name, password)	(String, String)=>Boolean	Creates user dir, password file, empty summaries file
login(name, password)	(String, String)=>Boolean	Reads password file and compares; prints error on mismatch
loadSummaries(name)	String=>List[String]	Reads Summaries.txt; returns non-empty lines as a list
saveSession(name, num, history, topics, mood)	...=>Unit	Calls ConversationBrain.saveChat then appends a summary line

3.7 Chatbot (chatBot.ui — ChatBotInteract.scala)

ChatBotInteract.scala

The core response-generation engine. Maintains the in-session conversation history as a mutable List and routes every user message to the correct handler. It is the only object that writes to history.

Member / Method	Signature / Type	Description
history (private var)	List[InteractionEntry]	Mutable in-session conversation log

setHistory(h) / getHistory	List[IE] <=> Unit	Allow Main to load a resumed session or retrieve for save
generateResponse(input, user)	(String, String) => (String, Option[String])	Master dispatcher: detects intent/sentiment/cuisine, routes to formatter, appends to history, returns (response, contextCuisine)

3.8 startChefBot (@main — CookingChatBot.scala)

CookingChatBot.scala

The application entry point. Orchestrates the full user journey: welcome banner, authentication loop, session-resume prompt, greeting with remembered preferences, main conversation loop, and final session-save with summary.

Member / Method	Signature / Type	Description
isValid(s)	String => Boolean	Returns true if string is non-empty and has no whitespace
readValid(prompt)	String => String	Recursive prompt loop — re-prompts until a valid non-whitespace input is given
startChefBot()	() => Unit	@main function wiring all components together

4. Built-in & Standard-Library Functions — Complete Index

The table below catalogues every Scala built-in, collection method, String method, Option combinator, and os-lib call used in the codebase, grouped by category.

4.1 Collection Operations

Function	Used In	Purpose
.map(f)	All files	Transforms every element of a List/Seq using function f
.filter(pred)	Chatbot, Engine	Keeps only elements satisfying predicate pred
.flatMap(f)	Brain, Chatbot	Maps then flattens; used to extract ingredients/tags across recipes
.find(pred)	Chatbot, Brain	Returns Option[A] — first element matching predicate or None
.exists(pred)	Brain, Chatbot	Returns true if any element satisfies pred
.forall(pred)	RecommendEngine	Returns true if all elements satisfy pred (used for None-safe filtering)
.count(pred)	Brain (mood)	Counts elements matching pred (counts pos/neg interactions)
.distinct	Brain, Chatbot	Removes duplicate elements from a list
.take(n)	Brain, Engine	Returns first n elements
.mkString(sep)	Formatter, Brain	Joins list elements into a single String with separator
.isEmpty / .nonEmpty	All files	Boolean checks on collections and Options
.size / .length	Auth, Chatbot	Returns the number of elements in a collection
.sortBy(f)	Brain (topics)	Sorts a list by key extracted by f
.zipWithIndex	Formatter, Main	Pairs each element with its 0-based index (for step numbering)
.toList	Brain, Prefs	Converts Seq / Array / os.ReadLines to immutable List
.toMap	PreferenceManager	Converts a List[(K,V)] into a Map[K,V]
List(...)	Recipes	Constructs an immutable List literal
Nil	Chatbot, Brain	Empty List constant; used to initialise history
:+	Chatbot	Appends one element to the end of a List (produces new list)
++	Brain	Concatenates two Lists
Random.shuffle(l)	RecommendEngine	Returns a randomly reordered copy of the list (scala.util.Random)
.headOption	Chatbot	Returns Option of first element; None if list is empty
.split(regex,-1)	Brain (loadChat)	Splits string by regex; -1 keeps trailing empty strings
.flatten	Chatbot	Flattens a List[Option[A]] or List[List[A]] into List[A]

<code>.distinct after++</code>	Brain suggestions	Deduplicates merged autocomplete candidates
--------------------------------	-------------------	---

4.2 String Operations

Function	Used In	Purpose
<code>.toLowerCase</code>	All files	Case-insensitive comparison; normalises user input
<code>.toUpperCase</code>	Formatter (guide)	Not used directly but <code>.capitalize</code> is
<code>.capitalize</code>	Brain, Chatbot	Uppercases first character of a String
<code>.trim</code>	Prefs, Brain, Main	Strips leading/trailing whitespace from strings
<code>.contains(sub)</code>	Brain, Chatbot	True if the string contains substring <code>sub</code>
<code>.startsWith(pre)</code>	Brain (prefix)	True if string starts with prefix <code>pre</code> (autocomplete)
<code>.mkString</code>	Formatter	Joins a collection of strings (see Collections)
<code>.nonEmpty</code>	Main, Prefs	True if String length > 0
<code>.equalsIgnoreCase</code>	Chatbot, Engine	Case-insensitive string equality check
<code>.indexOf(ch)</code>	PreferenceManager	Finds position of ':' separator in preference lines
<code>.substring(s,e)</code>	PreferenceManager	Extracts key and value from a preference line
<code>s"...\${expr}"</code>	All files	String interpolation — embeds Scala expressions in strings
<code>""" ...""".stripMargin</code>	Formatter, Main	Multi-line string with leading ' ' stripped
<code>.split("\\s+")</code>	Brain (smart search)	Tokenises input on whitespace to get keyword array
<code>.toInt</code>	Brain (loadChat)	Parses a numeric String to Int (session sequence numbers)
<code>.map(_ .trim)</code>	Prefs, Brain	Trims each string in a collection
<code>.filter(_ .contains)</code>	Prefs	Keeps only lines that contain the ':' separator
<code>.isWhitespace</code>	Main (isValid)	Character predicate used to reject blank-space-only inputs

4.3 Option Combinators

Function	Used In	Purpose
<code>.orElse(opt)</code>	RecommendEngine	Returns first Some; falls back to second Option if None
<code>.forall(pred)</code>	RecommendEngine	True if None OR if Some(v) satisfies pred (safe filter with no pref set)
<code>.foreach(f)</code>	Chatbot	Executes side-effect f if Some; no-op if None
<code>.map(f)</code>	Brain, Chatbot	Transforms the value inside Some; passes None through
<code>.isDefined / .isEmpty</code>	Chatbot	Pattern tests; <code>.isDefined == isSome</code>

.get	Chatbot (unsafe)	Extracts value from Some — used only after .find guard
Some(v)	Brain, Chatbot	Constructs a non-empty Option
None	All files	Empty Option constant
Option.match	Chatbot (contextCuisine)	Pattern matching on Some/None for branching logic
scala.util.Try(...).toOption	Main	Converts a potentially-throwing expression to Option

4.4 I/O & OS Operations (os-lib + StdIn)

Function / Method	Used In	Purpose
os.pwd	Auth, Brain, Prefs	Current working directory as an os.Path
os.pwd / "dir"	Auth, Brain, Prefs	Path concatenation operator — builds sub-paths
os.exists(path)	Auth, Brain, Prefs	Returns true if the path exists on disk
os.makeDir.all(path)	UserAuth	Creates directory and all parents (mkdir -p)
os.write(path, data)	UserAuth	Writes String to a new file (fails if exists)
os.write.over(path,d)	Brain, Prefs	Overwrites file content; creates file if absent
os.write.append(path,d)	UserAuth	Appends a line to an existing file
os.read(path)	UserAuth	Reads entire file as a String
os.read.lines(path)	Brain, Prefs	Reads file as an IndexedSeq[String] (one per line)
StdIn.readLine()	Main	Reads one line from stdin; returns null on EOF
print(s) / println(s)	Main, Auth	Writes to stdout without / with trailing newline

4.5 Control Flow & Pattern Matching

Construct	Used In	Purpose
match / case	Chatbot, Main	Scala's exhaustive pattern matching
case i if cond	Chatbot (generateResponse)	Guard patterns — conditional case branches
if / else if / else	All files	Standard conditional branching
while (cond) { }	Main	Mutable loop for auth retry and chat loop
return	Auth, Brain, Main	Early return from a method
for { } yield	(implicit in map/flatMap)	Not used directly; map/flatMap used instead
@main def	CookingChatBot	Scala 3 entry-point annotation
try { } catch { }	(implicit in Try)	scala.util.Try wraps toInt for safe parse

List.flatten (Some)	Chatbot	Flattens List[Option[String]] removing Nones
---------------------	---------	--

5. Data Flow Diagram

The diagram below illustrates how a single user message travels through the system from raw stdin input to a formatted response with personalised suggestions.

Step	Action	Component / Detail
1	User types a message	stdin via StdIn.readLine() in startChefBot
2	Input validation	isValid() and readValid() ensure no blank/whitespace input
3	generateResponse(input, user)	Chatbot object — master dispatcher in ChatBotInteract.scala
4	Sentiment & intent analysis	ConversationBrain.isNegativeSentiment() + detectIntent()
5	Context detection	detectCuisine() / detectTag() / detectDifficulty()
6	Preference update	PreferenceManager.storePref() / clearPref() — written to disk
7	Route to handler	Pattern match: help / summary / topics / mood / negative / recipe / search
8	Format response	ResponseFormatter methods produce the final string
9	History append	InteractionEntry added to Chatbot.history in memory
10	Suggestions	RecommendationEngine.recommend() reads prefs + contextCuisine
11	Output to user	println() in startChefBot prints response and suggestions
12	Session save (on exit)	ConversationBrain.saveChat() + UserAuth.saveSession()

6. Key Design Patterns

Singleton Object (object keyword)

Every major component (ConversationBrain, PreferenceManager, RecommendationEngine, ResponseFormatter, UserAuth, Chatbot) is a Scala singleton object. This avoids the need for explicit instantiation and ensures a single shared instance per JVM.

Immutable Data + Functional Pipelines

Recipe data is stored in immutable List[Recipe]. Filtering, mapping, and grouping operations return new collections rather than mutating in place. Only Chatbot.history (var) and filesystem state are mutable.

Pattern Matching as Router

Chatbot.generateResponse uses a large match/case block with guard conditions (case i if ...) to cleanly route user intent to the correct handler — a functional alternative to if/else chains or strategy objects.

Progressive-Relaxation Filtering

RecommendationEngine.recommend tries the most specific filter combination first (cuisine + tag + difficulty), then relaxes constraints in order until a non-empty result is found. This guarantees the user always receives suggestions.

Option-Guarded Filtering with forall

Helper predicates in RecommendationEngine use Option.forall: when a preference is None the predicate is vacuously true, so the filter is effectively skipped without any explicit if/else branching.

Persistent Key-Value Store via Plain Text

PreferenceManager simulates a key-value database using a plain text file with key:value lines. loadPrefs reads the whole file, the operation updates the Map, and write.over rewrites the entire file — a simple but effective pattern for small user data.

Recursive Input Loop (readValid)

The readValid helper in CookingChatBot is tail-recursively structured: it re-calls itself on invalid input rather than looping with a while. This keeps invalid-input handling concise and stateless.